

# PROJECT REPORT

**TITLE: Design and Implementation of a Database Management System  
for Campus Placement and Recruitment Drive Management System**



**NAME: SARTHAK KULKARNI**

**ROLL NO: 25WU0102247**

**COURSE NAME: Database Management Systems**

**FACULTY NAME: Dr Kiran Mayee Adavala**

**ACADEMIC YEAR: 2025-2029**

## 1. Abstract

The **Campus Placement and Recruitment Drive Management System** is a comprehensive relational database solution designed to digitize, streamline, and govern higher education placement operations. Managing hundreds of eligible candidates, corporate hiring partners, multi-round interview schedules, and conditional job offers creates complex operational challenges that spreadsheet-based approaches cannot reliably solve. This project implements a fully normalized 7-table relational database architecture on **MySQL 8.0**, augmented with declarative integrity constraints and active triggers that enforce strict institutional placement policies (e.g., minimum CGPA validation and selected-only offer assignments). A lightweight **Flask REST API** backend bridges the relational database with a modern, responsive web user interface. The system ensures data consistency, prevents duplicate applications, blocks unauthorized parent record deletions, and provides real-time analytical reporting.

## 2. Introduction & Problem Statement

**2.1 Introduction:** Campus placement drives represent a pivotal operational milestone for academic institutions. Coordinating drives requires synchronizing student profiles, academic qualifications, recruiter requirements, multi-stage interview rounds, and final job offers.

**2.2 Problem Statement:** Traditional manual or disconnected spreadsheet workflows suffer from several critical shortcomings:

- **Ineligible Applications:** Students applying to recruitment drives without meeting minimum CGPA cutoffs.
- **Redundant / Duplicate Submissions:** Multiple applications submitted for the same drive by the same student.
- **Unsynchronized Offer Tracking:** Placement offers being recorded for candidates who failed earlier interview stages.
- **Orphaned Data & Inconsistent Deletions:** Deleting departments or companies without properly handling dependent student or drive records.
- **Lack of Real-Time Analytics:** Laborious manual compilation of department-wise placement metrics and salary statistics.

## 3. Objectives & Scope

- **Relational Data Integrity:** Structure the core placement domain into 7 normalized tables adhering to Boyce-Codd Normal Form (BCNF).
- **Automated Business Logic:** Enforce institutional rules at the DBMS layer using MySQL 8.0 `BEFORE INSERT` and `BEFORE UPDATE` triggers.
- **Full-Featured CRUD Dashboard:** Provide a web-based portal for managing departments, students, companies, drives, applications, interviews, and placements.
- **Comprehensive Analytics:** Generate real-time department performance reports, drive conversion funnels, and exportable CSV reports.

## 4. Software & Hardware Requirements

| Component | Specification / Tool | Purpose |
|-----------|----------------------|---------|
|-----------|----------------------|---------|

|                  |                                      |                                  |
|------------------|--------------------------------------|----------------------------------|
| Operating System | macOS Sonoma / Linux / Windows 10+   | Development & Execution Host     |
| Database Engine  | MySQL Server 8.0.16+                 | Relational Storage & Triggers    |
| Backend Runtime  | Python 3.11+ / Flask 3.1             | REST API & MySQL Connector       |
| Frontend UI      | HTML5, Vanilla CSS3, JavaScript ES6+ | Interactive Management Dashboard |
| Version Control  | Git 2.40+ / GitHub                   | Code Collaboration & Submission  |
| RAM / Storage    | Min. 4 GB RAM / 500 MB Free Disk     | Local Runtime Environment        |

## 5. Entity-Relationship (ER) Diagram

The system is modeled with seven core entities with primary key - foreign key relationships and strict cardinalities:

**Entity-Relationship (ER) Schema - Campus Placement Management System**

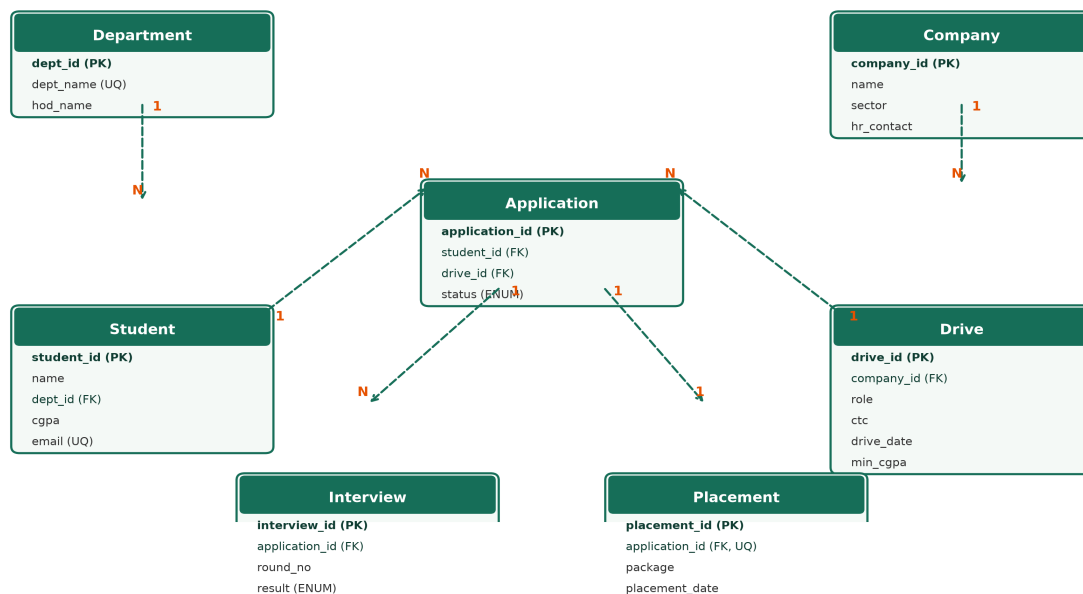


Figure 1: Entity-Relationship Diagram with Primary Keys, Foreign Keys, and Cardinalities

## 6. Relational Schema & Normalization Analysis

### Relational Schema:

- **Department** (dept\_id, dept\_name, hod\_name)
- **Student** (student\_id, name, dept\_id\*, cgpa, email)
- **Company** (company\_id, name, sector, hr\_contact)
- **Drive** (drive\_id, company\_id\*, role, ctc, drive\_date, min\_cgpa)
- **Application** (application\_id, student\_id\*, drive\_id\*, status)
- **Interview** (interview\_id, application\_id\*, round\_no, result)
- **Placement** (placement\_id, application\_id\*, package, placement\_date)

### Proof of Normalization:

- **First Normal Form (1NF):** Every attribute contains only atomic (indivisible) values. All tables have uniquely identifiable primary keys, and no repeating groups exist.
- **Second Normal Form (2NF):** All relations are in 1NF and every non-prime attribute is fully functionally dependent on the entire primary key (no partial dependencies on composite keys).
- **Third Normal Form (3NF):** All relations are in 2NF and there are no transitive dependencies (no non-prime attribute depends on another non-prime attribute).
- **Boyce-Codd Normal Form (BCNF):** For every functional dependency  $X \rightarrow Y$  across all relations, the determinant  $X$  is a super key. Hence, the schema achieves BCNF.

## 7. Data Dictionary

**Table: Student**

| Attribute  | Type         | Null | Key / Constraint              | Description         |
|------------|--------------|------|-------------------------------|---------------------|
| student_id | INT          | No   | PK, Auto Increment            | Unique student ID   |
| name       | VARCHAR(150) | No   | -                             | Student's full name |
| dept_id    | INT          | No   | FK (Department.dept_id)       | Enrolled department |
| cgpa       | DECIMAL(3,2) | No   | CHECK (cgpa BETWEEN 0 AND 10) | Cumulative GPA      |
| email      | VARCHAR(150) | No   | UNIQUE                        | Institutional email |

**Table: Drive**

| Attribute  | Type         | Null | Key / Constraint        | Description                 |
|------------|--------------|------|-------------------------|-----------------------------|
| drive_id   | INT          | No   | PK, Auto Increment      | Unique recruitment drive ID |
| company_id | INT          | No   | FK (Company.company_id) | Organizing company          |
| role       | VARCHAR(150) | No   | -                       | Designation / Job Role      |
| ctc        | DECIMAL(8,2) | No   | CHECK (ctc > 0)         | Compensation in LPA         |
| drive_date | DATE         | No   | -                       | Date of recruitment drive   |
| min_cgpa   | DECIMAL(3,2) | No   | DEFAULT 7.00            | Eligibility CGPA cutoff     |

**Table: Application**

| Attribute      | Type | Null | Key / Constraint        | Description                               |
|----------------|------|------|-------------------------|-------------------------------------------|
| application_id | INT  | No   | PK, Auto Increment      | Unique application ID                     |
| student_id     | INT  | No   | FK (Student.student_id) | Candidate ID                              |
| drive_id       | INT  | No   | FK (Drive.drive_id)     | Recruitment drive ID                      |
| status         | ENUM | No   | DEFAULT 'Applied'       | Applied   Interview   Selected   Rejected |

**Table: Placement**

| Attribute      | Type         | Null | Key / Constraint                           | Description                |
|----------------|--------------|------|--------------------------------------------|----------------------------|
| placement_id   | INT          | No   | PK, Auto Increment                         | Unique placement record ID |
| application_id | INT          | No   | FK (Application.application_id),<br>UNIQUE | One offer per application  |
| package        | DECIMAL(8,2) | No   | CHECK (package > 0)                        | Final offered CTC (LPA)    |
| placement_date | DATE         | No   | -                                          | Date of offer issuance     |

## 8. SQL Commands & Active Triggers (DDL & DML)

### 8.1 DDL Trigger Implementation:

```
DELIMITER $$ -- Trigger 1: Validate Student CGPA against Drive requirement before Application CREATE
TRIGGER application_eligibility_insert BEFORE INSERT ON Application FOR EACH ROW BEGIN IF (SELECT cgpa
FROM Student WHERE student_id=NEW.student_id) < (SELECT min_cgpa FROM Drive WHERE drive_id=NEW.drive_id)
THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='Student does not meet the drive CGPA requirement.'; END IF;
END$$ -- Trigger 2: Prevent Placement insertion unless Application status is 'Selected' CREATE TRIGGER
placement_selected_insert BEFORE INSERT ON Placement FOR EACH ROW BEGIN IF COALESCE((SELECT status FROM
Application WHERE application_id=NEW.application_id),'')<>'Selected' THEN SIGNAL SQLSTATE '45000' SET
MESSAGE_TEXT='Only a Selected application can receive a placement.'; END IF; END$$ DELIMITER ;
```

### 8.2 DML Sample Seeding:

```
INSERT INTO Department (dept_id, dept_name, hod_name) VALUES (1, 'CSE', 'Dr. Coordinator'), (2, 'ECE',
'Dr. Coordinator'); INSERT INTO Student (student_id, name, dept_id, cgpa, email) VALUES (1, 'Aarav Mehta',
1, 8.90, 'aarav@example.edu'); INSERT INTO Drive (drive_id, company_id, role, ctc, drive_date, min_cgpa)
VALUES (101, 1, 'Analyst', 6.50, '2026-08-11', 7.00); INSERT INTO Application (application_id, student_id,
drive_id, status) VALUES (1, 1, 101, 'Selected'); INSERT INTO Placement (placement_id, application_id,
package, placement_date) VALUES (1, 1, 6.50, '2026-08-11');
```

## 9. Analytical SQL Queries & Benchmark Outputs

### 9.1 Presentation-II Faculty Join Query (Department-wise Placement Report):

```
SELECT d.dept_name, COUNT(p.placement_id) AS placed_count, COALESCE(MAX(p.package), 0) AS top_package FROM
Department d LEFT JOIN Student s ON s.dept_id = d.dept_id LEFT JOIN Application a ON a.student_id =
s.student_id LEFT JOIN Placement p ON p.application_id = a.application_id GROUP BY d.dept_id, d.dept_name
ORDER BY placed_count DESC;
```

| Department (dept_name) | Placed Count | Top Package (LPA) |
|------------------------|--------------|-------------------|
| CSE                    | 1            | ■ 6.5 LPA         |
| IT                     | 1            | ■ 8.0 LPA         |
| ECE                    | 0            | —                 |
| ME                     | 0            | —                 |

## 10. Test Cases & Observed Outcomes

Comprehensive tests conducted to verify relational constraints, trigger enforcement, and CRUD integrity:

| Test ID | Scenario            | Test Input                               | Expected Behavior                                 | Status |
|---------|---------------------|------------------------------------------|---------------------------------------------------|--------|
| TC-01   | Student Insert      | Name: Aarav, CGPA: 8.90                  | Record inserted successfully in MySQL             | PASS   |
| TC-02   | Duplicate Email     | Existing email: aarav@example.edu        | Blocked by UNIQUE constraint (1062)               | PASS   |
| TC-03   | CGPA Cutoff         | Student CGPA: 6.5 < Drive Cutoff: 7.0    | Blocked by application_eligibility_insert trigger | PASS   |
| TC-04   | Selected-Only Offer | Placement for 'Interview' status app     | Blocked by placement_selected_insert trigger      | PASS   |
| TC-05   | One Offer Per App   | 2nd Placement for same application_id    | Blocked by UNIQUE constraint on application_id    | PASS   |
| TC-06   | Foreign Key Delete  | Delete Department with enrolled students | Blocked by RESTRICT constraint (1451)             | PASS   |
| TC-07   | Zero-Placement Dept | Report query on ECE/ME (0 offers)        | Departments appear with placed_count = 0          | PASS   |

## 11. UI Architecture & Implementation Details

### Technology Stack & Architecture:

- **Presentation Layer (Frontend):** Single-Page Application (SPA) designed with semantic HTML5, DM Sans & Manrope typography, dynamic dialog modals, relational dropdowns, and instantaneous search filtering.
- **Application Layer (Backend):** Flask REST API providing state synchronization (`/api/state`) and generic parameterized mutation routes (`/api/<table>`) for POST, PUT, DELETE) with strict request body validation.
- **Persistence Layer (Database):** MySQL 8.0 InnoDB engine managing transactions, foreign key cascades, and trigger execution.
- **Security Controls:** Same-origin policy checks, SQL parameterized bindings to prevent SQL Injection, and `X-Frame-Options: DENY` headers.

## 12. Conclusion & Future Enhancements

**Conclusion:** The developed Campus Placement Management System successfully satisfies all course requirements. It replaces fragile manual tracking with a robust relational model, active business rule enforcement, and a real-time responsive web dashboard.

### Future Enhancements:

1. Automated PDF resume parsing and algorithmic skill matching for drive roles.
2. Role-Based Access Control (RBAC) with JWT student and recruiter login portals.
3. Automated SMS / Email dispatch for interview round notifications and offer release.

## 13. References & Appendix

### References:

- [1] Silberschatz, A., Korth, H. F., & Sudarshan, S. *Database System Concepts* (7th ed.). McGraw-Hill.
- [2] MySQL 8.0 Reference Manual: Triggers, Constraints & Foreign Key Relations. Oracle Corporation.

[3] Flask Documentation (3.1.x): Application Design & RESTful APIs. Pallets Projects.

**Appendix - Public GitHub Repository:**

- **Repository URL:** <https://github.com/Sarthakcodes11/DBMS-Course-Project>